# Gestión de Estados de Cita

Backend ConfirmaMed • Guía de integración para Frontend

API de citas: estados, transiciones, endpoints e historial

Versión 1.1 • 15 de octubre de 2026 • Documento generado sobre la base de datos **develop**. Todos los cambios de estado quedan registrados en un historial auditable.

## 1. En qué consiste

Cada cita (appointment) ahora tiene un **estado explícito** que describe en qué punto de su ciclo de vida se encuentra. Antes el estado se infería de dos banderas (`isOccuped` / `isApproved`); ahora hay un campo **state** con código, nombre y color, pensado para mostrarse directamente en la UI.

Además, el backend registra **cada** cambio de estado en un historial (quién lo hizo, por qué y cuándo), y un proceso automático mueve las citas por el tiempo (empieza / termina) sin intervención del usuario.

**Importante para el frontend:** usa siempre `state.code` para la lógica y `state.name` / `state.color` para mostrar. Los campos `isOccuped` e `isApproved` siguen presentes por compatibilidad, pero quedarán obsoletos.

## 2. Los estados

| code | Nombre | Color | Significado |
|---|---|---|---|
| `libre` | Libre | #22c55e | Estado inicial de toda cita. El cupo está disponible para asignarse a un paciente. |
| `asignada` | Asignada | #3b82f6 | Ya tiene un paciente. Es el estado donde vive la cita hasta que llega su hora. |
| `en_atencion` | En atención | #f59e0b | La cita está transcurriendo (llegó la hora de inicio). Lo pone el sistema automáticamente. |
| `finalizada` | Finalizada | #6b7280 | La cita terminó (pasó la hora de fin). Lo pone el sistema automáticamente. |
| `cancelada` | Cancelada | #ef4444 | La cita fue cancelada. El cupo NO se reutiliza; queda como registro. |
| `no_asistio` | No asistió | #a855f7 | Durante la atención el paciente no se conectó. Se marca desde un endpoint. |

## 3. Transiciones permitidas

Cualquier transición no listada se rechaza con **409 Conflict**. El backend valida el estado de origen en la misma operación de base de datos, así que es seguro ante choques de concurrencia.

| De | A | Quién lo dispara | Condición |
|---|---|---|---|
| `libre` | `asignada` | Endpoint `POST /assign` | La cita sigue libre |
| `asignada` | `en_atencion` | Sistema (automático) | Llegó start_hour |

| De | A | Quién lo dispara | Condición |
|---|---|---|---|
| `asignada` | `cancelada` | Endpoint `POST /{id}/cancel` | Estado = asignada |
| `en_atencion` | `finalizada` | Sistema (automático) | Pasó end_hour |
| `en_atencion` | `no_asistio` | Endpoint `POST /{id}/no-show` | Estado = en_atencion |

**Flujo normal:** libre → asignada → en_atencion → finalizada.

**Nota sobre pagos:** por ahora toda cita `asignada` pasa a `en_atencion` cuando llega su hora. Cuando exista el módulo de pagos se añadirá la regla: una cita asignada no pagada se cancelará automáticamente al llegar la hora.

## 4. Formato general de respuesta

Todos los endpoints responden con un sobre estándar. El recurso va dentro de `items`. Las llaves JSON usan **camelCase**.

```
{
  "success": true,
  "statusCode": 200,
  "message": "Recurso obtenido satisfactoriamente",
  "path": "/api/appointments/279",
  "responseTime": "2026-10-09T22:36:52.8918Z",
  "items": { ...recurso o arreglo... }
}
```

### Objeto Appointment (items)

Este es el objeto que devuelven assign, cancel, no-show y getById. En los listados, `items` es un arreglo de este mismo objeto (sin `videoCallLink`).

```
{
  "id": 279,
  "dateAppointment": "2026-10-20",
  "startHour": "09:00:00",
  "endHour": "09:30:00",
  "duration":   { "id": 2, "interval": "00:30:00" },
  "doctor":     { "id": 5, "name": "Ana",  "lastName": "Rojas", "document": "123" },
  "speciality": { "id": 3, "name": "Cardiologia", "code": "CAR" },
  "patient":    { "id": 3, "name": "Luis", "lastname": "Perez", "document": "999" },
  "status": true,
  "isOccuped": true,          // LEGACY - no usar para logica nueva
  "isApproved": true,         // LEGACY - no usar para logica nueva
  "state": {                  // <-- USAR ESTE
     "id": 2,
     "code": "asignada",
     "name": "Asignada",
     "color": "#3b82f6"
  },
  "userId": 1,
  "roomName": "cm-279",
  "roomUrl": "https://....daily.co/cm-279",
  "roomCreatedAt": "2026-10-09T22:30:00",
  "videoCallLink": "https://....daily.co/cm-279?t=TOKEN"
}
```

Ojo con dos detalles reales del JSON: en **doctor** la llave es `lastName` (N mayúscula) y en **patient** es `lastname` (n minúscula). `patient` es `null` si la cita está libre.

## 5. Endpoints

Ruta base: `/api/appointments` (sin distinción de mayúsculas). Los endpoints marcados como **Auth** requieren header `Authorization: Bearer <jwt>`. Los marcados **Anónimo** no requieren token.

**POST** `/api/appointments/assign`

**Auth:** Anónimo • Asigna un paciente a una cita libre. Provisiona la videollamada y envía el correo de confirmación. Devuelve la cita en estado `asignada`.

### Request body

```
{
  "appointmentId": 279,
  "patientId": 3
}
```

### Respuesta (200)

El objeto Appointment con `state.code = "asignada"` y `videoCallLink` poblado.

### Errores

| Código | Cuándo |
|---|---|
| **409** | La cita no está libre, o alguien la tomó primero (carrera). |
| **404** | La cita no existe. |

**POST** `/api/appointments/{id}/cancel` **[NUEVO]**

**Auth:** Auth • Cancela una cita **asignada**. El cupo no se reutiliza. Libera la sala de video. Queda registrada en el historial con el usuario autenticado y la nota.

### Parámetros

| Parámetro | En | Tipo | Descripción |
|---|---|---|---|
| `id` | ruta | int | Id de la cita |

### Request body

```
{
  "note": "El paciente pidió cancelar"   // opcional
}
```

### Respuesta (200)

El objeto Appointment con `state.code = "cancelada"`.

### Errores

| Código | Cuándo |
|---|---|
| **409** | La cita no está en estado 'asignada'. |
| **404** | La cita no existe. |

**POST** `/api/appointments/{id}/no-show` **[NUEVO]**

**Auth:** Auth • Marca que el paciente no asistió. Solo aplica a citas **en atención**. Queda registrada en el historial. Reachable una vez que el proceso automático pone la cita en en_atencion.

## Parámetros

| Parámetro | En | Tipo | Descripción |
|---|---|---|---|
| `id` | ruta | int | Id de la cita |

## Request body

```
{
  "note": "El paciente no se conectó"   // opcional
}
```

## Respuesta (200)

El objeto Appointment con `state.code = "no_asistio"`.

## Errores

| Código | Cuándo |
|---|---|
| **409** | La cita no está en estado 'en_atencion'. |
| **404** | La cita no existe. |

**GET** `/api/appointments/dates/{dateSelected}/filters`

**Auth:** Auth • Lista las citas de una fecha, con filtros. **Nuevo:** filtro por estado.

## Parámetros

| Parámetro | En | Tipo | Descripción |
|---|---|---|---|
| `dateSelected` | ruta | date (yyyy-MM-dd) | Fecha a consultar |
| `state` | query | string | Filtra por código de estado, p.ej. 'libre' (NUEVO) |
| `specialityId` | query | int? | Filtra por especialidad |
| `doctorId` | query | int? | Filtra por doctor |
| `isOccuped` | query | bool? | LEGACY - filtra por ocupada |
| `limit / offset` | query | int? | Paginación (default 10 / 0) |

## Respuesta (200)

Arreglo de objetos Appointment.

**GET** `/api/appointments/user/dates/{dateSelected}/filters`

**Auth:** Auth • Citas del doctor del usuario autenticado para una fecha. **Nuevo:** si mandas `state` filtra por ese estado; si no lo mandas, por regla de negocio devuelve solo citas ocupadas.

## Parámetros

| Parámetro | En | Tipo | Descripción |
|---|---|---|---|
| `dateSelected` | ruta | date | Fecha |
| `state` | query | string | Filtra por código de estado, p.ej. 'libre' (NUEVO) |
| `specialityId` | query | int? | Filtra por especialidad |
| `limit / offset` | query | int? | Paginación |

### Respuesta (200)

Arreglo de objetos Appointment.

**GET** `/api/appointments/patient/{patientId}`

**Auth:** Auth • Historial de citas de un paciente.

### Parámetros

| Parámetro | En | Tipo | Descripción |
|---|---|---|---|
| `patientId` | ruta | int | Id del paciente |
| `specialityId` | query | int? | Filtra por especialidad |
| `startDate` | query | date? | Desde esta fecha |
| `limit / offset` | query | int? | Paginación |

### Respuesta (200)

Arreglo de objetos Appointment.

**GET** `/api/appointments/{id}`

**Auth:** Anónimo • Obtiene una cita por id (incluye su `state`).

### Parámetros

| Parámetro | En | Tipo | Descripción |
|---|---|---|---|
| `id` | ruta | int | Id de la cita |

### Respuesta (200)

Un objeto Appointment.

### Errores

| Código | Cuándo |
|---|---|
| **404** | La cita no existe. |

## Endpoints de videollamada (contexto)

Relacionados con el flujo pero sin cambios en esta feature: `POST /{id}/video/provision`, `GET /{id}/video/doctor-token`, `DELETE /{id}/video`. Al cancelar, finalizar o marcar no-asistió, el backend libera la sala automáticamente.

## 6. Proceso automático (scheduler)

Un proceso en el backend corre **cada minuto** y, usando la hora de Colombia (America/Bogota):

| Transición | Cuándo | Efecto adicional |
|---|---|---|
| asignada → en_atencion | Llega la hora de inicio (start_hour) | — |
| en_atencion → finalizada | Pasa la hora de fin (end_hour) | Libera la sala de video |

**Qué significa para el frontend:** el estado de una cita puede cambiar solo, sin que el usuario haga nada. No asumas que una cita asignada seguirá asignada; vuelve a consultarla (o refresca la vista) para reflejar `en_atencion` / `finalizada`. El mismo proceso envía los recordatorios por correo (24 h y 2 h antes) para las citas asignadas.

## 7. Historial de estados

Cada cambio de estado genera un registro con: estado de origen y destino, **razón**, **usuario** que lo hizo (o *sistema* si fue automático), **nota** opcional y fecha. Razones posibles:

| Razón | Origen |
|---|---|
| `asignacion` | Al asignar un paciente |
| `cancelacion` | Cancelación manual (incluye la nota) |
| `no_asistio` | Marca de no asistió (incluye la nota) |
| `inicio_automatico` | El sistema pasó la cita a en_atencion |
| `finalizacion_automatica` | El sistema finalizó la cita |
| `migracion` | Carga inicial de datos existentes |

Nota: hoy el historial se usa internamente para auditoría. Si el frontend necesita mostrarlo (una línea de tiempo por cita), se puede exponer un endpoint `GET /api/appointments/{id}/history`; queda como trabajo pendiente a solicitud.

## 8. Manejo de errores en la UI

| HTTP | Significado | Sugerencia de UI |
|---|---|---|
| **409** | Transición no válida para el estado actual (o cita tomada por otro) | Mostrar el mensaje del backend y refrescar la cita |
| **404** | La cita no existe | Volver al listado / refrescar |
| **400** | Request inválido (validaciones) | Mostrar el mensaje de error |
| **401** | No autenticado | Redirigir a login |

El cuerpo de error mantiene el mismo sobre: `success=false`, `statusCode` y `message` con el texto a mostrar.

## 9. Checklist de migración para el frontend

- ■ Leer el estado desde `state.code` y pintar con `state.name` / `state.color`.
- ■ Dejar de depender de `isOccuped` / `isApproved` (quedarán obsoletos).
- ■ Habilitar el botón Cancelar solo cuando `state.code == 'asignada'`.
- ■ Habilitar Marcar no-asistió solo cuando `state.code == 'en_atencion'`.
- ■ Manejar 409 refrescando la cita (el estado pudo cambiar).

- Refrescar las vistas periódicamente: el scheduler cambia estados solo.
- Usar el filtro `?state=libre` para los cupos disponibles (en ambos listados).